



Innovation Centre for Education



Yenepoya Institute Of Arts, Science, Commerce & Management

Project – High Level Design

on

Cloud Cost Intelligence Platform For Finance

Student Name: KITHU MANOJ

Industry Mentor: Mr Sumit Kumar

Guided by: Ms Manjula

Table of Contents

1. Introduction.	2
1.1. Scope of the document.	
1.2. Intended Audience	
1.3. System overview.	
2. System Design.	3
2.1. Application Design	
2.2. Process Flow.	
2.3. Information Flow.	
2.4. Components Design	
2.5. Key Design Considerations	
3. Data Design.	6
3.1. Data Model	
3.2. Data Access Mechanism	
3.3. Data Retention Policies	
3.4. Data Migration	
4. Interfaces	8
5. State and Session Management	9
6. Caching	10
7. Non-Functional Requirements	11
7.1. Security Aspects	
7.2. Performance Aspects	
8. References	12

1.INTRODUCTION

1.1. Scope Of The Document

This document provides a high-level design of the Cloud Cost Intelligence Platform for Finance. It outlines the system architecture, major components, data flow, and integration points required to analyze, monitor, and optimize cloud expenses. The document serves as a blueprint for developers, stakeholders, and project managers.

1.2 Intended Audience

- Software Developers
- Data Engineers
- Financial Analysts
- Project Managers
- Cloud Architects

1.3 System Overview

The Cloud Cost Intelligence Platform is designed to help organizations analyze and optimize cloud expenditures. It processes billing data from cloud providers or sample datasets and converts it into meaningful insights.

Key Features:

- Cost tracking by service, region, and time
- Monthly and yearly cost trend analysis
- Visualization of spending patterns
- Identification of high-cost services

Technology Stack:

- Backend: Python (Pandas)
- Frontend: Power BI
- Data Source: Excel / CSV files

2. SYSTEM DESIGN

2.1 Application Design

The application follows a layered modular architecture to ensure flexibility and scalability.

Layers:

1. Data Ingestion Layer
 - Accepts input from Excel/CSV files
 - Validates file format and schema
2. Data Processing Layer
 - Cleans data (removes nulls, duplicates)
 - Standardizes formats (date, currency)
3. Analytics Layer
 - Aggregates cost data
 - Performs grouping (monthly, service-wise)
 - Detects anomalies
4. Visualization Layer

Displays charts and dashboards and Provides user-friendly insights

2.2 Process Flow

The system operates in the following sequence:

- User uploads billing dataset
- Data ingestion module reads the file
- Data cleaning and preprocessing is performed

Cost calculations are executed:

- Total cost
- Monthly cost
- Service-wise cost

Processed data is passed to visualization

Dashboard displays charts and summaries

2.3 Information Flow

Input Flow:

User → Upload File → System

Processing Flow:

Raw Data → Cleaned Data → Aggregated Data

Output Flow:

Processed Data → Charts → Dashboard

2.4 Components Design

1. Frontend Component

- Built using Power BI
- Displays charts (pie charts, bar graphs)
- Provides user interaction

2. Backend Component

- Python scripts handle data processing
- Performs calculations and analysis

3. Data Storage Component

- Temporary storage in memory (Pandas DataFrame)
- Optional persistent storage (SQLite)

The system is composed of several key components that work together to achieve the desired functionality. The frontend component provides the user interface for uploading data and viewing results. It is designed to be simple and user-friendly.

The backend component is responsible for data processing and analysis. It is implemented using Python and handles all computational tasks. The analytics engine is a critical part of the backend, performing data aggregation and generating insights.

The data storage component is used to store datasets and processed results. In the basic version, data is stored in memory using Pandas DataFrames, but it can be extended to use databases such as SQLite for persistent storage.

2.5 Key Design Considerations

Several factors were considered during the design of the system. Scalability is important to ensure that the system can handle large datasets without performance degradation. Accuracy is critical since the system deals with financial data, and even minor errors can lead to incorrect insights.

Usability is another key consideration, as the platform is intended for users with varying levels of technical expertise. The system must provide a simple and intuitive interface for data upload and visualization.

Extensibility is also important, allowing the system to be enhanced in the future with additional features such as real-time data integration and advanced analytics.

3. DATA DESIGN

3.1 Data Model

Sample Schema:

- **Month**
- **Service Name** (Networking, Storage, etc.)
- **Region** (America, Europe, Asia)
- **Usage Hours**
- **Cost (USD)**

Relationships:

- One month → Multiple services
- One service → Multiple cost entries

The data model defines the structure of the input dataset and how it is organized within the system. Each record in the dataset represents a cloud usage entry, containing fields such as date, service name, region, usage quantity, and cost.

The data model is designed to support efficient querying and aggregation. For example, data can be grouped by date to calculate monthly costs or by service to identify high-cost components.

3.2 Data Access Mechanism

- Data is accessed using Pandas library
- Functions like: `read_excel()`, `read_csv()`
- Data stored in DataFrames for efficient processing

Data is accessed using Python libraries such as Pandas, which provide efficient methods for reading and processing structured data. Functions such as `read_excel()` and `read_csv()` are used to load data into DataFrames.

Once loaded, the data can be manipulated using various operations such as filtering, grouping, and aggregation. This approach ensures fast and efficient data processing.

3.3 Data Retention Policies

The system implements basic data retention policies to manage storage and ensure data privacy. Data is retained for a specific period, after which it can be archived or deleted.

Retention policies are designed to balance the need for historical analysis with storage constraints. Sensitive financial data is handled carefully to ensure confidentiality and compliance with data protection standards.

3.4 Data Migration

Data migration involves transferring data from one system or format to another. In the current implementation, data is manually uploaded by users. However, the system can be extended to support automated data ingestion from cloud providers.

Future enhancements may include scheduled data imports and integration with APIs, enabling real-time data updates and continuous monitoring.

4. INTERFACES

User Interface

- File upload interface
- Dashboard displaying:
 - Monthly cost trends
 - Service-wise distribution
 - Pie charts and bar graphs

System Interfaces

- Excel/CSV file input
- Integration with Power BI for advanced visualization

The system provides user and system interfaces. The user interface allows users to upload datasets and view results. It includes features such as file selection, data preview, and dashboard visualization.

System interfaces include integration with external tools such as Power BI. These interfaces enable the system to exchange data with other applications.

5. STATE AND SESSION MANAGEMENT

State and session management play a crucial role in defining how user interactions and application data are handled across different stages of system execution. In the Cloud Cost Intelligence Platform for Finance, the system is primarily designed using a stateless architecture, ensuring simplicity, scalability, and efficient processing of independent requests.

First, it simplifies system design by eliminating the need for complex session tracking mechanisms. Second, it enhances scalability, as the system can handle multiple user requests simultaneously without maintaining session-specific memory. Third, it improves reliability, as failures in one request do not affect others.

For example, when a user uploads a cloud billing dataset, the system processes the file, generates insights, and displays results without storing the user's session state. If the user uploads another dataset, it is treated as a completely new request.

While the current system operates without persistent sessions, future enhancements may introduce session management to improve user experience and functionality.

Session management would allow the system to:

- Maintain user-specific data across multiple interactions
- Enable user authentication and authorization
- Store user preferences and settings
- Allow users to revisit previously uploaded datasets and reports

In such a scenario, each user session would be assigned a unique session ID, which would be used to track user activities and maintain continuity.

6. CACHING

Caching is an important design consideration in the Cloud Cost Intelligence Platform for Finance, as it directly impacts system performance, responsiveness, and scalability. Caching refers to the temporary storage of frequently accessed or computationally expensive data so that it can be quickly retrieved without reprocessing.

In a system that performs repeated data analysis on large cloud billing datasets, caching plays a crucial role in reducing computation time and improving user experience.

The primary purpose of caching in this platform is to optimize performance by minimizing redundant computations. When users repeatedly request similar analytics—such as monthly cost summaries or service-wise breakdowns—the system can retrieve precomputed results from the cache instead of recalculating them.

This is particularly useful in scenarios where:

- The same dataset is analyzed multiple times
- Users frequently refresh dashboards
- Large datasets require significant processing time

By reducing repeated processing, caching helps in improving response time and overall system efficiency.

The caching strategy in this system is designed to balance performance improvement with memory usage. Since the current implementation is relatively simple, caching is primarily handled at the application level.

Key Strategies:

- **Result Caching:** Store computed outputs such as total cost, monthly summaries, and service-wise aggregations
- **Data Caching:** Store cleaned and processed datasets to avoid repeated preprocessing
- **Visualization Caching:** Cache chart-ready data for faster rendering

7. NON FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes and operational constraints of the Cloud Cost Intelligence Platform for Finance. Unlike functional requirements, which describe what the system does, non-functional requirements specify how the system performs under various conditions. These requirements are critical for ensuring system reliability, efficiency, scalability, and user satisfaction.

The platform is designed to handle financial data efficiently while maintaining high standards of performance, security, and usability.

7.1 Security Aspects

Performance is a crucial factor for the success of the system, especially when dealing with large cloud billing datasets. The platform must process data efficiently and provide results within an acceptable time frame.

The system is expected to:

- Process datasets containing thousands of records within a few seconds
- Perform data aggregation and analytics with minimal latency
- Generate dashboards and visualizations quickly

Efficient data processing is achieved using optimized Python libraries such as Pandas, which support vectorized operations and fast computations. Additionally, caching mechanisms are used to reduce redundant processing and improve response times.

The system should also be capable of handling multiple requests without significant degradation in performance.

7.2 Scalability Requirements

Scalability refers to the system's ability to handle increasing amounts of data and users without compromising performance. The platform is designed with scalability in mind to support future growth.

The current implementation supports small to medium datasets, but it can be extended to handle large-scale data by:

- Using distributed processing frameworks
- Integrating with cloud storage solutions
- Implementing distributed caching systems

The modular architecture ensures that individual components can be scaled independently. For example, the analytics engine can be optimized or replaced without affecting other parts of the system.

8. REFERENCES

google.com

wikipedia.com